Lecture 1
Linear Optimization

Wei Wang

College ISEE, Zhejiang University
Email: wangw@zju.edu.cn

Outline

1. Linear Optimization Problems
2. Simplex Method
3. Network Flow Problems

Linear Programming

- Minimize *linear* function over *linear* inequality and equality constraints:

$$\text{minimize } c^T x$$

$$\text{subject to } Gx \preceq h$$

$$Ax = b$$

- *Standard Form LP:*

$$\text{minimize } c^T x$$

$$\text{subject to } Ax = b$$

$$x \succeq 0$$

Transformation to Standard Form

- Introduce *slack varibles* s_i for inequality constraints:

$$\text{minimize } c^T x$$

$$\text{subject to } Gx + s = h$$

$$Ax = b$$

$$s \succeq 0$$

- Express x as difference between two nonnegative variables $x^+, x^- \succeq 0 : x = x^+ - x^-$

$$\text{minimize } c^T x^+ - c^T x^-$$

$$\text{subject to } Gx^+ - Gx^- + s = h$$

$$Ax^+ - Ax^- = b$$

$$x^+, x^-, s \succeq 0$$

A Two Variable Linear Program

$$\text{maximize } z = 3x + 5y$$

$$\text{subject to } 2x + 3y \leq 10$$

$$x + 2y \leq 6$$

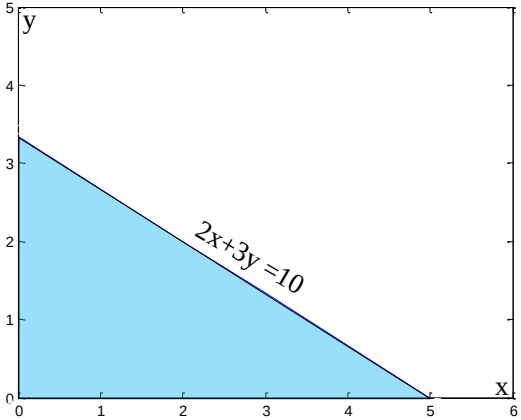
$$x \leq 4$$

$$y \leq 3$$

$$x, y \geq 0$$

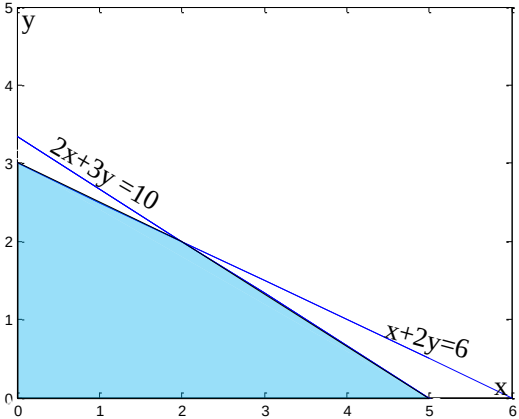
Geometrical Method

- Graph the constraints: $2x + 3y \leq 10$, $x, y \geq 0$



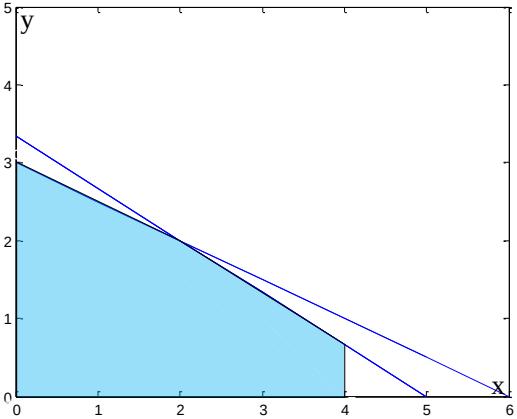
Geometrical Method

- Add the constraint: $x + 2y \leq 6$



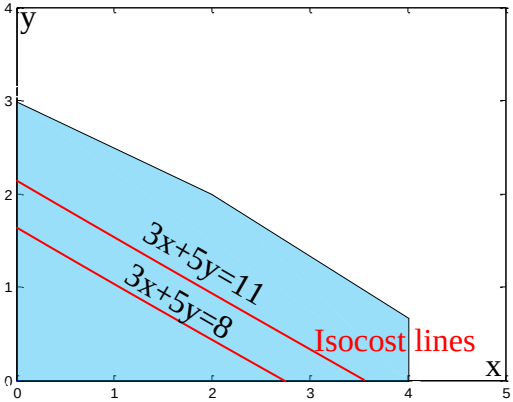
Geometrical Method

- Add the constraint: $x \leq 4$, $y \leq 3$ and obtain the feasible region



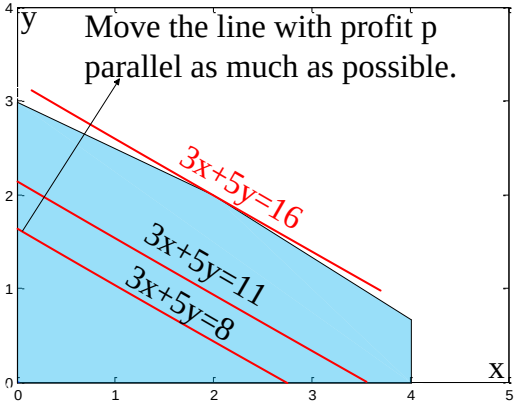
Geometrical Method

- Graph the points such that $3x + 5y = p$ for various values of p



Geometrical Method

- Find the maximum value of p such that there is a feasible solution with $3x + 5y = p$



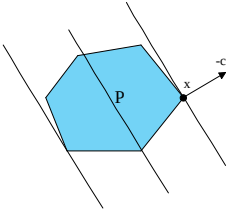
Extreme Point Property

Definition (Extreme Point)

x in polyhedron P is an extreme point (*corner point*) if there does not exist two other points $y, z \in P$ such that $x = \theta y + (1-\theta)z$ for some $\theta \in [0,1]$.

Theorem

Assume that a LP in standard form is feasible and the optimal objective value is finite. There exists an optimal solution which is an extreme point.



Algorithms

What if the number of dimensions is large?

- Simplex Method
- Interior-point Method
- Cutting-plane Method

Simplex method

- Move from one vertex to another without enumerating all the vertices.
- *Very efficient* in practice but specialized for LP

We will cover other algorithms for general convex optimization later.

Simplex Method

- Basic idea: to confine the search to *corner points* of the feasible region (of which there are only *finitely* many) in a most intelligent way.
- Steps:
  1. Find a starting extreme point.
  2. Move to a better extreme point.
  3. Judge whether the current solution is optimal and determine repeat/stop.

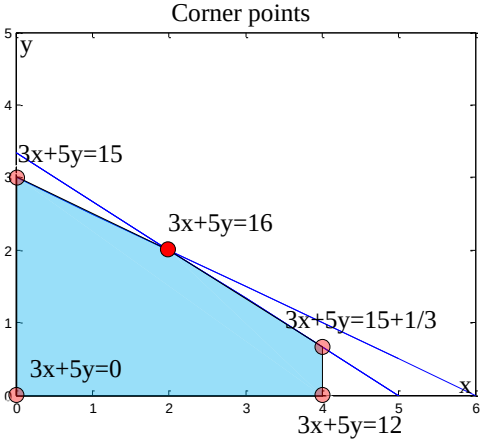
Simplex Method

- Basic idea: to confine the search to *corner points* of the feasible region (of which there are only *finitely* many) in a most intelligent way.
- Steps:
  1. Find a starting extreme point $\rightarrow$ Find an initial basic feasible solution.
  2. Move to a better extreme point $\rightarrow$ Pivot operation: A nonbasic column is transformed as a column of the matrix. The variable corresponding to the pivot column becomes a basic variable.
  3. Judge whether the current solution is optimal and determine repeat/stop. $\rightarrow$ Compute the reduced cost.

Simplex Method

Homework1:

Solve the example problem in Page 5 using simplex method with initial point 0.



Networks Flow Problems

Network (directed graph, digraph): m nodes connected by n directed arcs

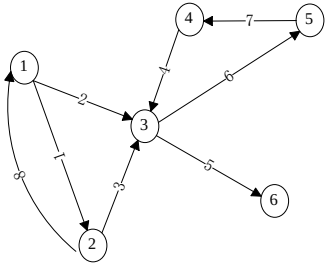
- Arcs are ordered pairs (i, j) of nodes.
- We assume there is at most one arc from node i to node j .
- There are no loops (arcs(i, i)).

Arc-node incidence matrix: $m \times n$ matrix A with entries:

$$A_{ij} = \begin{cases} 1 & \textit{if arc } j \textit{ starts at node } i \\ -1 & \textit{if arc } j \textit{ ends at node } i \\ 0 & \textit{otherwise} \end{cases}$$

Networks Flow Problems

$$A = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & -1 & -1 & -1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \end{bmatrix}$$



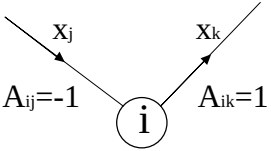
Networks Flow Problems

Flow vector $x \in R^n$:

- x_j: Flow (of material, traffic, information, . . .) through arc j.
- Positive if in direction of arc; negative otherwise.

Total flow leaving node i:

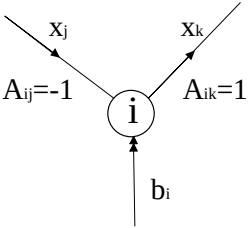
$$\sum_{j=1}^{n} A_{ij} x_j = (Ax)_i$$



External Supply

Supply vector $b \in R^m$:

- b_i is external supply at node i (negative b_i represents external demand).
- Must satisfy $1^T b = 0$ (total supply = total demand).



Balance equations :

$$Ax = b$$

Minimum Cost Network Flow Problem

$$\text{minimize } c^T x$$

$$\text{subject to } Ax = b$$

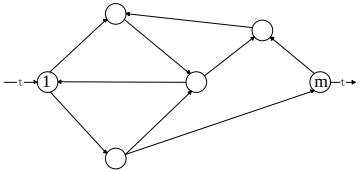
$$l \leq x \leq u$$

- c_i is unit cost of flow through arc i.
- l_j and u_j are limits on flow through arc j (typically, $l_j \leq 0$, $u_j \geq 0$).
- We assume $l_j < u_j$, but allow $l_j = -\infty$ and $u_j = \infty$.

Includes many network optimization problems as special cases.

Maximum Flow Problem

Maximize flow from node 1 (source) to node m (sink) through the network:



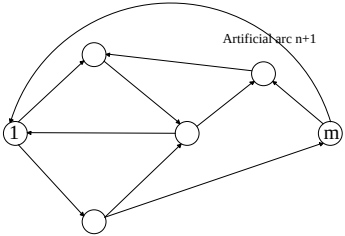
$$\text{maximize } t$$

$$\text{subject to } Ax = te$$

$$l \leq x \leq u$$

where $e = (1, 0, \ldots, 0, -1)$.

Formulation as Minimum Cost Flow Problem



$$\text{minimize } -t$$

$$\text{subject to } \begin{bmatrix} A & -e \end{bmatrix} \begin{bmatrix} x \\ t \end{bmatrix} = 0$$

$$l \leq x \leq u$$

Implications for Combinatorial Optimization

$$\text{minimize } c^T x$$

$$\text{subject to } Ax = b$$

$$l \leq x \leq u$$

$$x \in Z^n$$

- An integer linear program, very difficult in general.
- Equivalent to its linear program relaxation

$$\text{minimize } c^T x$$

$$\text{subject to } Ax = b$$

$$l \leq x \leq u$$

if A is totally unimodular and b, l, u are integer vectors (extreme optimal solution of the relaxation is optimal for the integer LP).

Shortest Path Problem

Shortest path in directed graph with node-arc incidence martix A:

- (Forward) Paths from node 1 to m can be represented by vectors x with

$$Ax = (1, 0, \ldots, 0, -1),\ x \in \{0, 1\}^n$$

- Shortest path is solution of

$$\begin{aligned} &\text{minimize } 1^T x \\ &\text{subject to } Ax = (1, 0, \ldots, 0, -1) \\ &x \in \{0, 1\}^n \end{aligned}$$

LP formulation:

$$\begin{aligned} &\text{minimize } 1^T x \\ &\text{subject to } Ax = (1, 0, \ldots, 0, -1) \\ &0 \leq x \leq 1 \end{aligned}$$

Network Simplex Method

- Basic idea: Construct negative-cost cycles based on a spanning tree of the network graph.
- Example

$$\text{minimize } c^T x$$

$$\text{subject to } Ax = b$$

$$x \geq 0$$

Network Simplex Method

- Steps:
  1. Find an initial feasible spanning tree.
  2. Add an arc $\overline{e}$ into the tree such that the created simple cycle has a negative cost.
  3. Increase as much flow as possible along the simple cycle without violating the constraints.
  4. Remove one of the arcs of the cycle e to obtain a new spanning tree $\overline{T}$.
  5. Judge whether the current tree is optimal and determine repeat/stop.

$$\overline{T} = T + \overline{e} - e$$

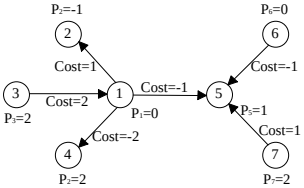
Network Simplex Method

- Key issues:
  1. Begin with an initial feasible spanning tree. $\rightarrow$ How to determine the initial tree?
  2. Add an arc $\overline{e}$ into the tree such that the new induced simple cycle has a negative cost. $\rightarrow$ How to choose the in-arc?
  3. Increase the flow along the simple cycle as much as possible without violating the constraints. $\rightarrow$ What if the cost does not strictly decrease?
  4. Discard one of the arcs of the cycle e to obtain a new spanning tree. $\overline{T}$ $\rightarrow$ How to choose the out-arc?
  5. Judge whether the current tree is optimal and determine repeat/stop. $\rightarrow$ How to judge the optimility?

Network Simplex Method

Choose in-arcs based on their prices

- Determine the price vector $\mathbf{p}$.
  - Select p_r arbitrarily.
  - Set the prices of the outward neighbors j with $(r,j) \in T$ to $p_j = p_r - c_{rj}$ and those of the inward neighbors j with $(j,r) \in T$ to $p_j = p_r + c_{jr}$.
  - Repeat the process with the neighbors j replacing r.



- Reduced cost of arc (i,j):

$$r_{ij} = c_{ij} + p_j - p_i$$

Network Simplex Method

Choose out-arcs based on their updated flows

- The maximum increment by which the flow of all arcs of C^+ can be increased and the flow of all arcs of C^- can be decreased

$$\delta = \min_{(i,j)\in C^-} x_{ij}$$

- The updated flow vector

$$\overline{x}_{ij} = \begin{cases} x_{ij} & (i,j) \notin C \\ x_{ij} + \delta & (i,j) \in C^+ \\ x_{ij} - \delta & (i,j) \in C^- \end{cases}$$

- Discard one of the arcs with $\overline{x}_{ij} = 0$

Network Simplex Method

Definition (Strongly Feasible Tree)

A feasible tree T with corresponding flow vector $\mathbf{x}$ is said to be strongly feasible if every arc (i,j) of T with $x_{ij} = 0$ is oriented away from the root.

- The generated strongly feasible trees are distinct such that a tree will not be infinitely repeated.
- Define the join of C as the first node of C that lies on the unique simple path of T that starts from the root and ends at $\bar{i}$ which is the start point of the in-arc. Choose the candidate out-arc encoutered first as C is traversed in the direction of the in-arc $\bar{e}$ starting from the join node.

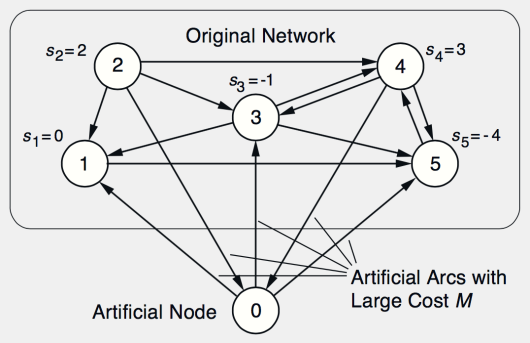
Algorithm Termination

- Two cases of algorithm termination:
  - Optimal if all reduced costs are nonnegative.
  - Unbounded if all arcs in the cycle have the same orientation with the in-arc $\bar{e}$.

Network Simplex Method

Obtain an initial strongly feasible tree (Big-M Method)

- Introduce an extra node labeled 0 and having zero supply $s_0 = 0$.
- Add a set of artificial arcs consisting of an arc $(i, 0)$ for each node i with $s_i > 0$ and an arc $(0, i)$ for each node i with $s_i \leq 0$. The cost coefficient of all these arcs is M.



Network Simplex Method

- M is large enough which satisfies

$$2M > \sum_{(i,j)\in P^+} c_{ij} - \sum_{(i,j)\in P^-} c_{ij}$$

for all simple path P of the original graph, where P^+ and P^- are the sets of forward and backward arcs of P, respectively.

A Shortest Path Example

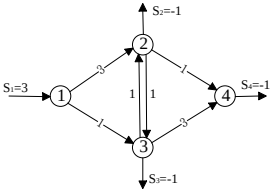
The corresponding minimum cost flow problem is

$$\text{minimize } \sum_{(i,j)\in A} c_{ij} x_{ij}$$

$$\text{subject to } \sum_{j|(1,j)\in A} x_{1j} - \sum_{j|(j,1)\in A} x_{j1} = 3$$

$$\sum_{j|(i,j)\in A} x_{ij} - \sum_{j|(j,i)\in A} x_{ji} = -1$$

$$0 \le x_{ij}, \forall (i,j) \in A$$



A Shortest Path Example

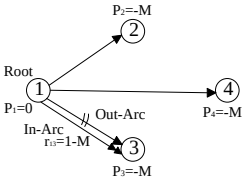
- Select the origin node 1 as root. Introduce the artificial arcs connecting the origin with each node $i \neq 1$ with very large cost M
- Use as an initial tree the set of artificial arcs with root node the origin. The initial tree is strongly feasible (all arcs are oriented away from the root).
- The corresponding price vector is $(0, -M, -M, -M)$ and the associated reduced costs of the nonartificial arcs are

$$r_{1j} = c_{1j} - M, \forall (1,j) \in A$$

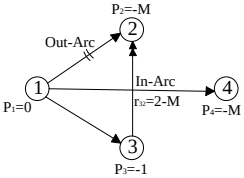
$$r_{ij} = c_{ij}, \forall (i,j) \in A, i \neq 1, j \neq 1$$

- One possible outcome of the first iteration is to select some arc $(1,j) \in A$ as in-arc, and to select the artificial arc connecting 1 and j as out-arc. The process will then be continued with the flow and price vectors corresponding to the new tree.

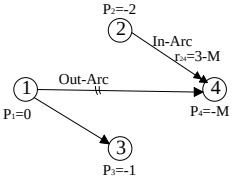
A Shortest Path Example



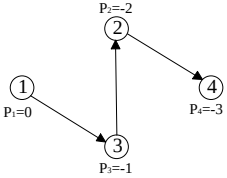
1st Pivot



2nd Pivot

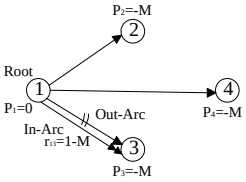


3rd Pivot

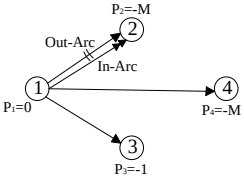


Final Tree

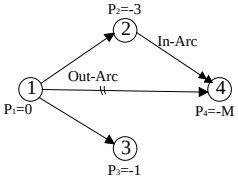
A Shortest Path Example



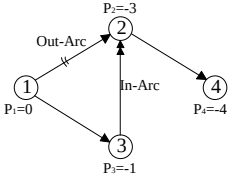
1st Pivot



2nd Pivot



3rd Pivot



Final Tree

A Shortest Path Example

1. Each artificial arc eventually becomes the out-arc but never becomes the in-arc.
2. In all trees, all the arcs are oriented away from the origin and carry unit flow.
3. We use the rule that the in-arc is an arc with minimum reduced cost. As a result, there are exactly $N-1$ pivots, and each time the out-arc is an artificial arc. In this case the simplex method works exactly like Dijkstra's algorithm.

It can be shown that observations (1) and (2) above hold in general for the simplex method applied to feasible shortest path problems, and observation (3) also holds in general provided $c_{ij} \geq 0$ for all arcs (i,j).

D. P. Bertsekas, *Network Optimization: Continuous and Discrete Models, Athena Scientific*, 1998.

Summary

- What is linear optimization?
- How to solve linear optimization?
  - Graphical method
  - Simplex method
- Application: Network flow problem